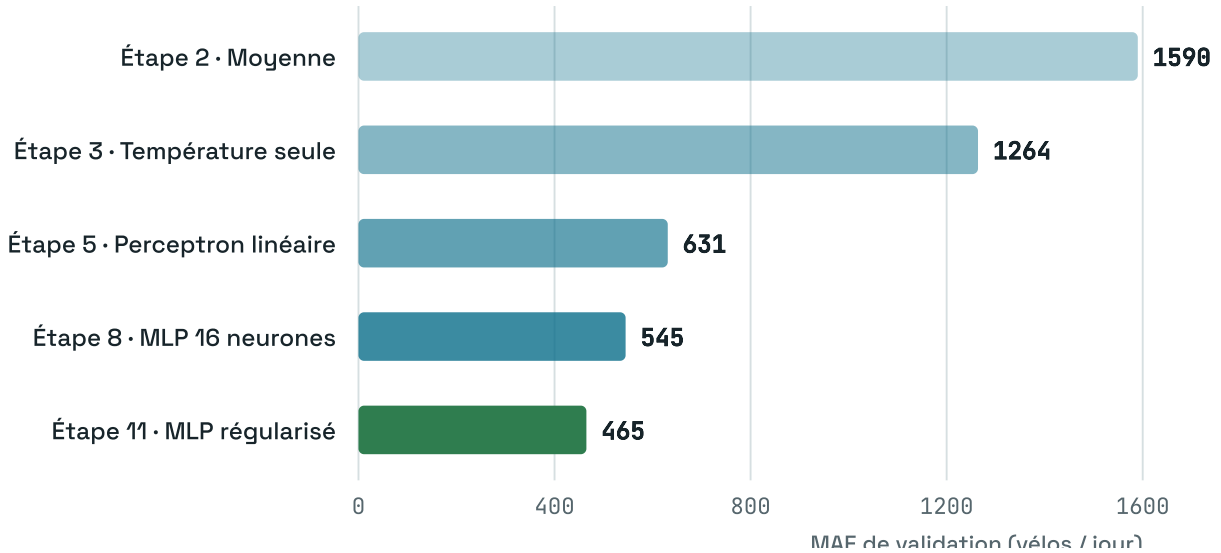
Notebook 11 à l'oral

Les 12 étapes du notebook `11_tests_simples_day.ipynb`, une par une : ce qu'on fait, la formule clé, le résultat, et le message à faire passer.

Problème. Prédire le nombre de vélos loués par jour (`cnt`) à partir de la météo, de la saison et du calendrier. 731 jours, 2011 et 2012.

Démarche. Partir du modèle le plus simple et ajouter une idée à chaque étape. Tout est codé à la main en numpy pour montrer ce que Keras fait derrière `model.fit()` .



Fil rouge de la présentation : l'erreur moyenne (MAE de validation, en vélos par jour) à chaque étape.

1 Préparer les données

Objectif : donner au modèle des entrées propres et sur la même échelle.

- On retire `casual` et `registered` : leur somme vaut `cnt` , ce serait une **fuite de données**.
- On encode `season` et `weathersit` en **one-hot** (une colonne 0/1 par valeur), car « hiver = 4 » n'est pas un nombre qui a un sens. On obtient 14 variables.
- On découpe en **train / validation / test** (439 / 146 / 146 jours).
- On **normalise** avec la moyenne et l'écart-type du train seulement, et on exprime `cnt` en milliers.

$$x^{\text{norm}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

> MESSAGE Sans normalisation, une seule valeur de learning rate ne peut pas convenir à toutes les variables.

2 Baseline : prédire la moyenne

Objectif : avoir un point de comparaison.

On prédit chaque jour la moyenne du train (4503 vélos), sans regarder aucune variable. L'erreur est mesurée avec la **MAE**, lisible directement en vélos.

$$\text{MAE} = \frac{1}{n}\sum_i |\hat{y}_i - y_i|$$

1590 vélos d'erreur moyenne par jour. Tout modèle doit faire mieux.

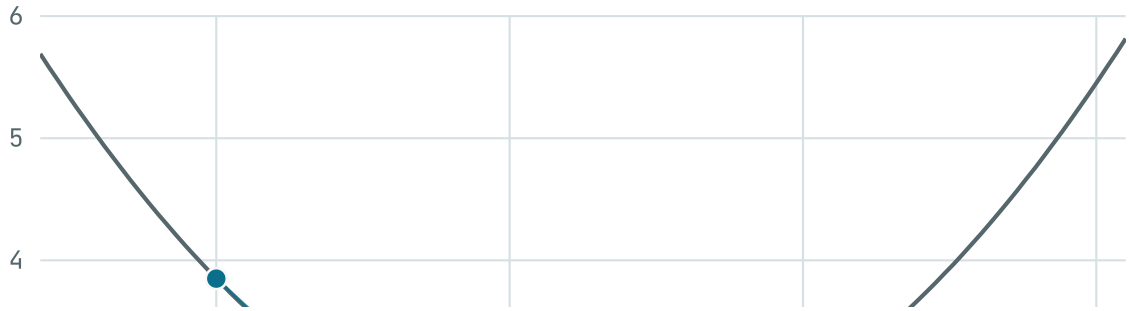
3 Un neurone, une variable : la descente de gradient

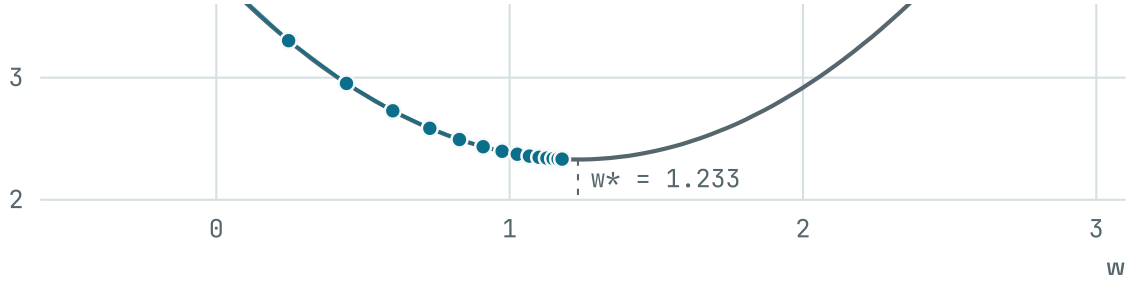
Objectif : comprendre la descente de gradient sur le cas le plus simple.

- Modèle : $\hat{y} = w\,x + b$ avec x = la température. C'est une droite.
- On mesure l'erreur avec la MSE, on calcule sa pente (le **gradient**) et on fait un petit pas dans le sens inverse. On répète 50 fois.

$$\frac{\partial L}{\partial w} = \frac{2}{n}\sum_i (\hat{y}_i - y_i)\,x_i \qquad w \leftarrow w - \eta\,\frac{\partial L}{\partial w}$$

Learning rate : 0.01 | 0.1 | 0.5 | 0.9 | 1.05 converge vite





La perte en fonction de w et les 15 premiers pas en partant de $w = 0$. Les pas raccourcissent près du minimum. Les boutons servent aussi pour l'étape 4.

1264 vélos On retrouve exactement la solution de `np.polyfit` ($w = 1.233$, $b = 4.503$) : la descente de gradient fonctionne.

> MESSAGE Pour un réseau de neurones, il n'y a pas de formule exacte : la descente de gradient est la seule méthode.

4 Tester le learning rate

Objectif : montrer que η est le réglage le plus sensible.

Même code, seul η change. À chaque pas, l'écart au minimum est multiplié par $1 - 2\eta$:

$$w_k - w^* = (1 - 2\eta)^k \, (w_0 - w^*)$$

- η = 0.001 ou 0.01 : très lent.
- η = 0.1 : converge en une dizaine d'époques.
- η = 0.5 : converge en **un seul pas**.
- η = 0.9 : oscille autour du minimum, puis converge.
- η = 1.05 : **diverge**, la perte explose.

> MESSAGE Trop petit, c'est lent ; trop grand, ça diverge. On teste des puissances de 10.

5 Perceptron linéaire : toutes les variables

Objectif : utiliser les 14 variables au lieu d'une seule.

Même principe, un poids par variable. En écriture matricielle, une ligne de code calcule toutes les prédictions et tous les gradients.

$$\hat{y} = Xw + b \qquad \nabla_w L = \frac{2}{n} X^\top (\hat{y} - y)$$

Les variables étant normalisées, on peut lire les poids : `yr` (le service grandit en 2012) et la température font monter les locations ; humidité, vent et pluie les font baisser.

631 vélos 2.5 fois mieux que la moyenne. Train et validation restent proches : pas de sur-apprentissage.

6 Batch, mini-batch ou SGD

Objectif : choisir sur combien d'exemples calculer chaque pas.

VARIANTE	EXEMPLES PAR PAS	PAS PAR ÉPOQUE	COMPORTEMENT OBSERVÉ
batch complet	439	1	lisse mais très lent
mini-batch	**32**	**14**	**rapide et lisse**
SGD	1	439	très rapide mais bruité

> MESSAGE Le mini-batch est le bon compromis : 32 est la valeur par défaut de Keras.

7 Le perceptron de Rosenblatt

Objectif : montrer le perceptron historique (1958) et sa limite.

- On transforme le problème en classification : le jour est-il « chargé » (au-dessus de la médiane) ?
- Le perceptron répond 1 si $x \cdot w + b > 0$, sinon 0. Il ne corrige ses poids que quand il se trompe.

$$w \leftarrow w + \eta\,(y - \hat{y})\,x$$

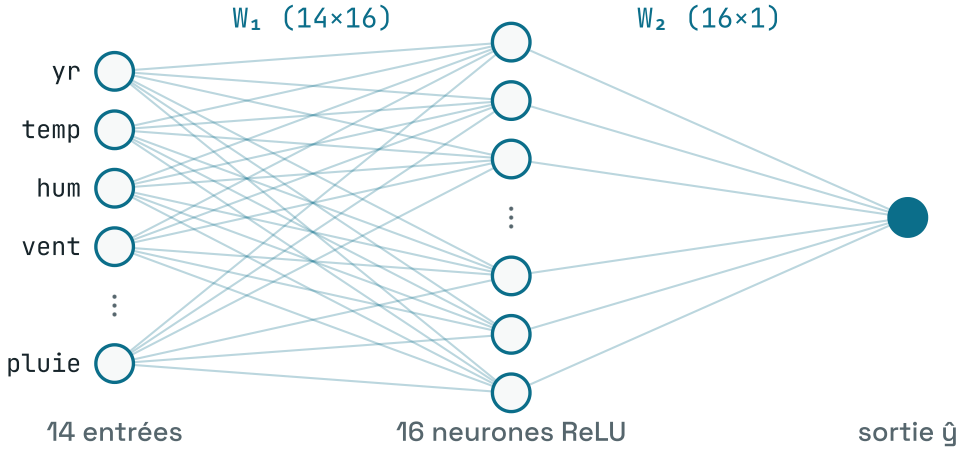
75 à 90 % de bonnes réponses, mais ça oscille sans fin : les jours proches de la médiane ne sont pas séparables par un hyperplan.

Solution : remplacer le seuil par une **sigmoïde** $\sigma(z) = 1/(1+e^{-z})$, dérivable, et entraîner par descente de gradient. L'accuracy monte alors régulièrement jusqu'à environ 88 % et se stabilise.

> MESSAGE Une activation dérivable permet la descente de gradient : c'est la base des réseaux modernes.

8 Le MLP et la rétropropagation

Objectif : capturer des effets non linéaires (par exemple, les locations plafonnent quand il fait très chaud).



$$H = \mathrm{ReLU}(XW_1 + b_1) \qquad \hat{y} = HW_2 + b_2$$

- $\mathbf{ReLU} = \mathbf{max}(0, z)$ rend le réseau non linéaire. Sans elle, deux couches linéaires équivalent à une seule.
- **Rétropropagation** : on part de l'erreur en sortie et on la fait remonter couche par couche (dérivation en chaîne). À chaque couche, gradient des poids = entrée$^\top$ × erreur.
- On vérifie les gradients par une dérivée numérique : les deux valeurs sont identiques (−0.85205672).

545 vélos Mieux que le perceptron linéaire, mais l'écart entre train (≈ 400) et validation se creuse : début du sur-apprentissage.

9 Combien de neurones cachés

Objectif : trouver la bonne capacité du modèle.

NEURONES	PARAMÈTRES	MAE TRAIN	MAE VALIDATION
1	17	585	611
16	257	406	545
64	**1025**	**352**	**482**
256	4097	291	552

> MESSAGE Plus de neurones améliore toujours le train, pas la validation. Avec 4097 paramètres pour 439 jours, le réseau mémorise.

10 Les optimiseurs

Objectif : accélérer et stabiliser l'apprentissage.

- **SGD :** $\theta \leftarrow \theta - \eta\, g$. Descente lente et régulière (559).
- **Momentum :** on garde 90 % du pas précédent, comme une bille qui prend de l'élan. Le plus rapide, mais sur-apprend tôt (614).
- **Adam :** momentum + un pas adapté à chaque poids. Démarre plus lentement, finit le plus bas (534).

$$v \leftarrow 0.9\, v - \eta\, g \qquad \theta \leftarrow \theta + v$$

> MESSAGE Adam est le choix par défaut : il est peu sensible au réglage du learning rate.

11 Régularisation : L2 et early stopping

Objectif : réduire le sur-apprentissage.

- **L2 :** on pénalise les grands poids. À chaque pas, les poids sont tirés vers 0, ce qui rend la fonction plus lisse.
- **Early stopping :** on garde le modèle de l'époque où la validation est la meilleure (ici l'époque 188).

$$L_{\text{total}} = \text{MSE} + \frac{\lambda}{2}\,\|W\|^2$$

λ	MAE TRAIN	MAE VALIDATION	ÉCART
0	277	590	313
0.01	**319**	**502**	**183**
0.1	485	517	32

465 vélos MLP 64 neurones + L2 ($\lambda = 0.01$) + early stopping : le meilleur modèle en validation.

12 Évaluation finale sur le test

Objectif : une note honnête, sur des jours jamais utilisés pour choisir.

On évalue le modèle retenu **une seule fois** sur le test.

499 vélos d'erreur moyenne par jour, soit environ 11 % de la moyenne journalière. Un peu plus que sur la validation (465), ce qui est normal : le modèle a été choisi sur la validation.

Conclusion à dire

- Avec un seul neurone et la descente de gradient, on divise déjà l'erreur par 2.5 par rapport à la moyenne.
- Une couche cachée avec ReLU capte les effets non linéaires et fait encore mieux.
- Avec seulement 439 jours d'entraînement, le réseau sur-apprend vite : la régularisation (L2, early stopping) est indispensable.
- Résultat final : erreur divisée par plus de 3, de 1590 à **499 vélos par jour**.

Questions possibles du jury

QUESTION	RÉPONSE COURTE
Pourquoi enlever `casual` et `registered` ?	Leur somme vaut `cnt` : c'est une fuite de données, inutilisable en production.

Pourquoi normaliser sur le train seulement ?	Sinon des informations de la validation et du test entrent dans l'entraînement.
Pourquoi un jeu de validation et un jeu de test ?	La validation sert à choisir les réglages ; le test donne une note non biaisée par ces choix.
Pourquoi ReLU ?	Sans activation non linéaire, empiler des couches reste un modèle linéaire.
Comment savoir que la rétropropagation est juste ?	On la compare à une dérivée numérique par différences finies : identiques.
Comment détecter le sur-apprentissage ?	La MAE de train baisse pendant que celle de validation stagne ou remonte.

Chiffres issus de l'exécution de `notebooks/11_tests_simples_day.ipynb` (graine 0, découpage 439 / 146 / 146).